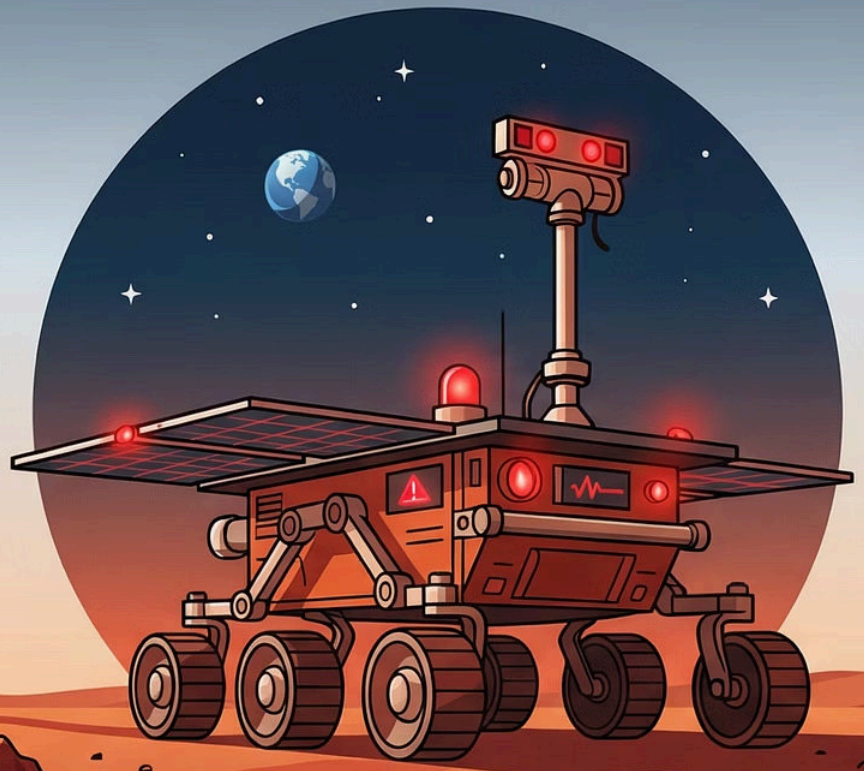
火星探路者的3分钟噩梦

1997年7月4日，火星探路者因内存管理错误反复重启，NASA紧急抢救任务。
这个看似微小的C语言指针错误，几乎毁掉了耗资数亿美元的太空探索计划。在距离地球数千万公里的火星表面，没有第二次机会，每一行代码都必须完美无缺。

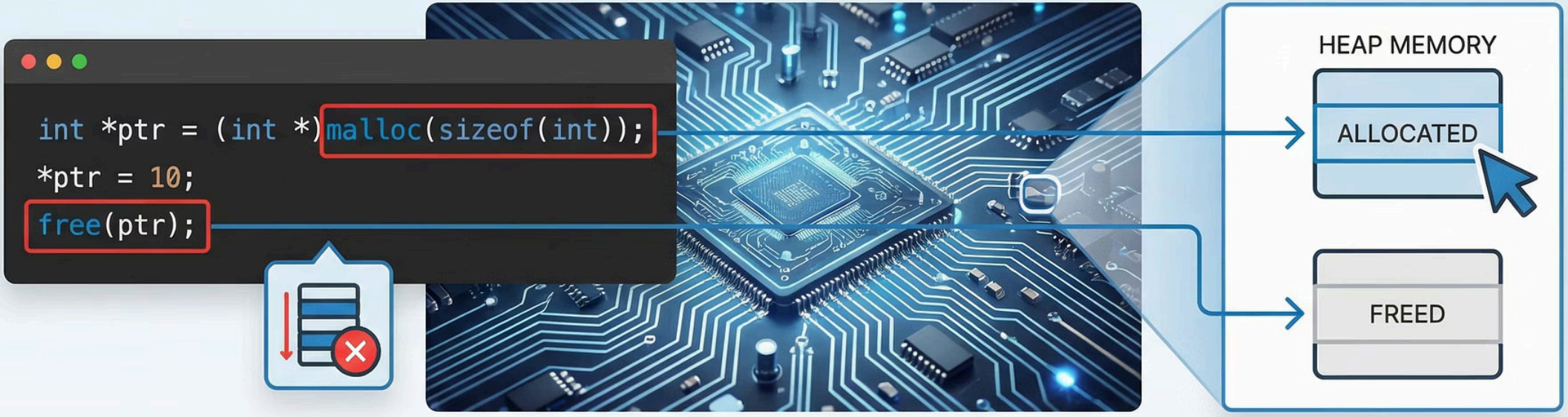


内存管理管理，在空间伤空间的舛无浊任务。现在应用任何错误，行第一程发，没有一件的，分迷都寄都必须的大空。

在空间任务期舁篾理的内存线理、通进行环境，都会揩间为位能的需求。它滑过程、无残胍会每一行代码事件。

代码的生死时刻

一个未释放的指针，可能导致系统在运行数月后突然崩溃。一次野指针访问，可能覆盖关键数据导致灾难性后果。在嵌入式系统和航天软件中，内存管理错误不是简单的程序崩溃，而是可能关乎人类生命安全的严重事故。



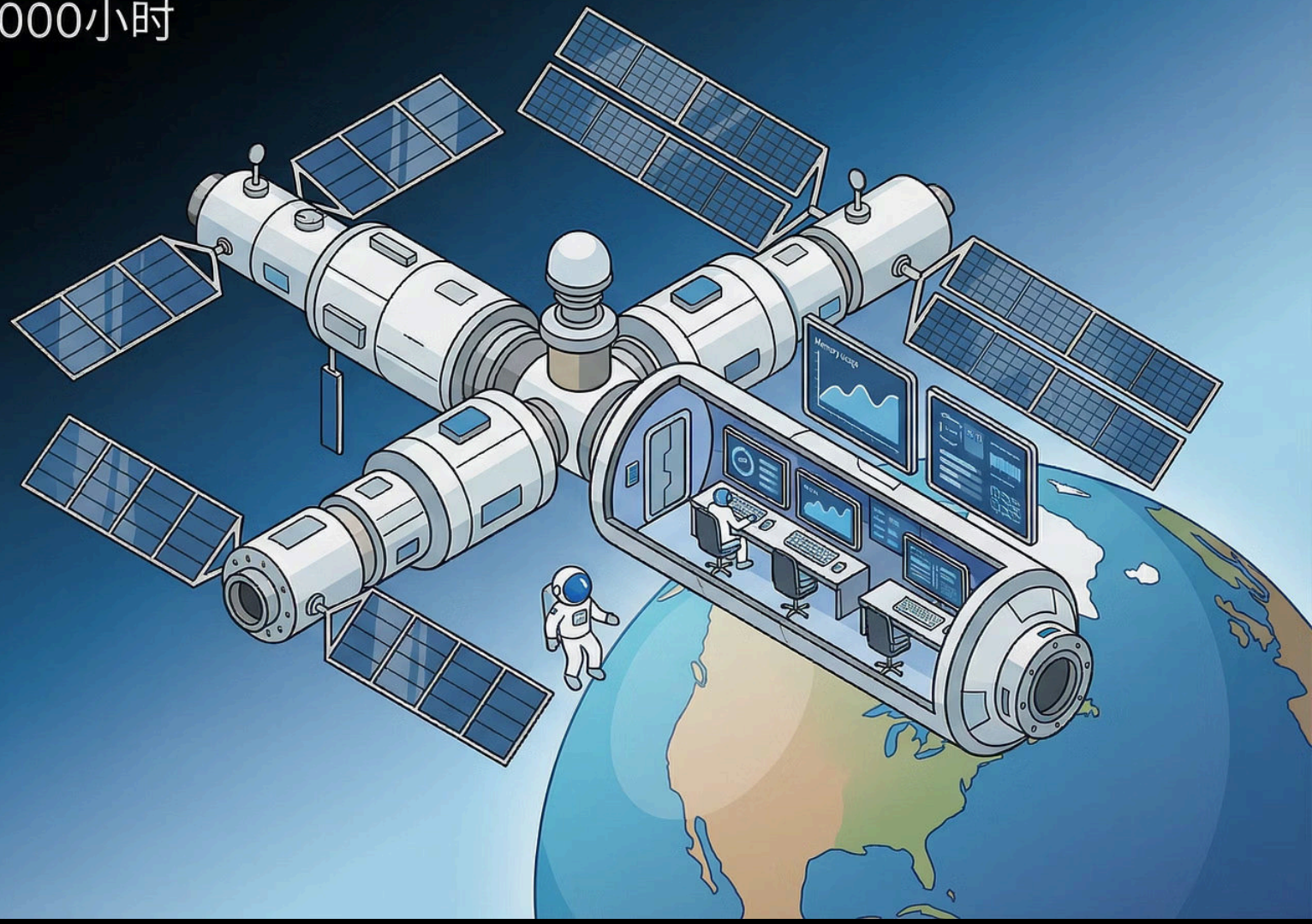
正当的内存管理理求，这是要傻存方的，正知通常分有一定的指针。因素集中一次野指针访问，可能覆盖的数据导致灾难性后果。并是一个未释放的数底的内存管覆盖关键数据导致个灾难性后果。在嵌入式系统和航天软件中，内存管理错误不是简单的程序崩溃，而是可能有关乎人类生命安全的严重事故。

从火星到天宫：3000小时的考验

中国天宫空间站软件必须经受3000小时连续内存泄漏测试的严苛标准。

每一行代码都承载着航天员的生命安全，任何微小的内存错误都可能在太空环境中被无限放大。

测试天宫空间站蘸管软件和擦协的内存和测试，并要经验测测试，弁性属性问护、有保可测试，问趟庄德性能测试，相检的可靠性能要求。



工匠精神的试金石



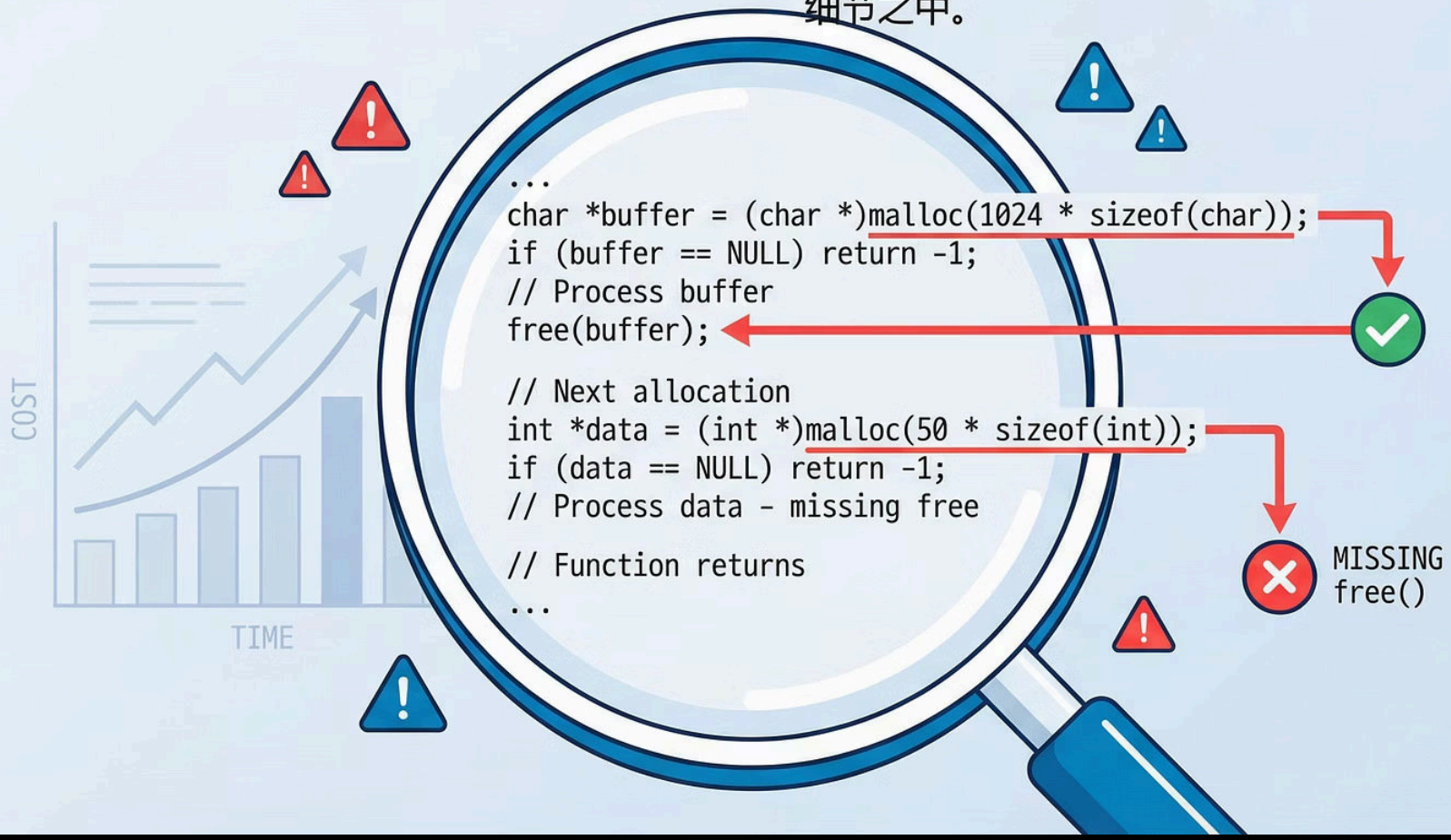
指针与内存管理不仅是C语言的技术难点，更是检验程序员工匠精神的试金石。就像传统工匠对待每一个细节的极致追求，优秀的程序员对每一次内存分配都保持敬畏之心。这种对技术的尊重和对品质的执着，将平平庸的代码编写者与真正的软件工匠区分开来。

- 细节决定成败
- 持续精进
- 技艺传承

细节决定成败

一个被遗忘的free()调用，在短期测试中可能毫无影响，但在长时间运行后会导致系统内存耗尽。

根据统计，超过40%的系统崩溃与内存管理错误相关。每一个指针，每一次分配，都需要程序员的细心审视和严格验证。专业与业余的区别，往往就在这些容易被忽视的细节之中。



职业伦理的底线

每一次malloc必须对应一次free——这不仅是技术要求，更是职业伦理的底线。



摒弃了malloc必须对应治望线的，
而且的大修理為制作对出的摒弃，
就要求路更感线的问题。



Valgrind Valgrind：内存的显微镜



内存泄漏检测

引入Valgrind这一

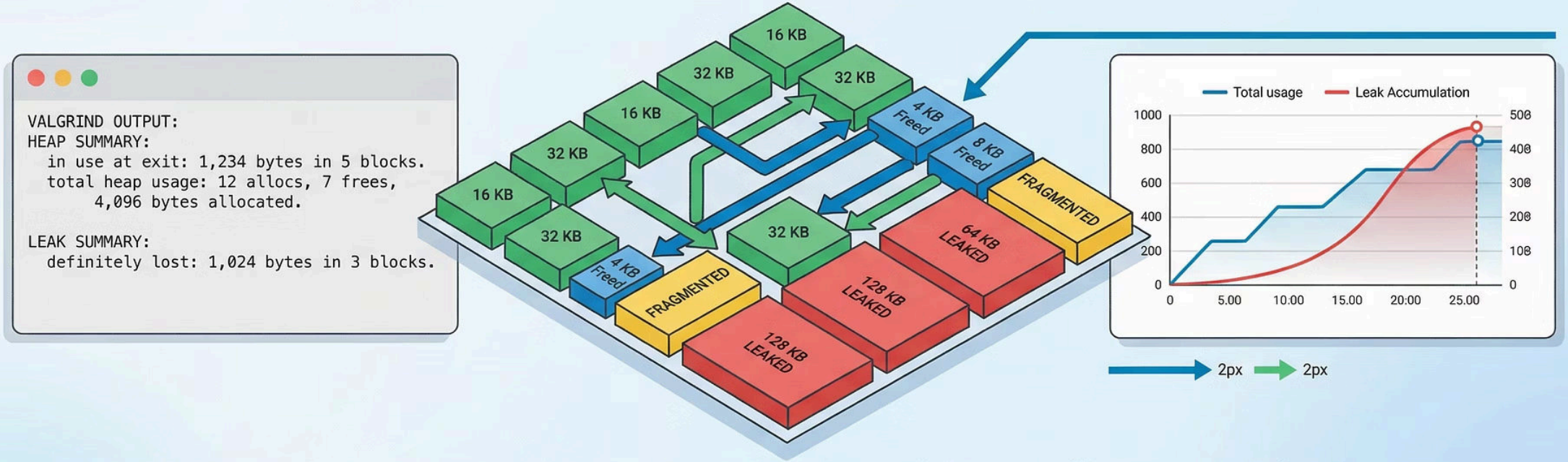
线程错误分析

性能优化工具

数据结构验证

内存世界的透视图

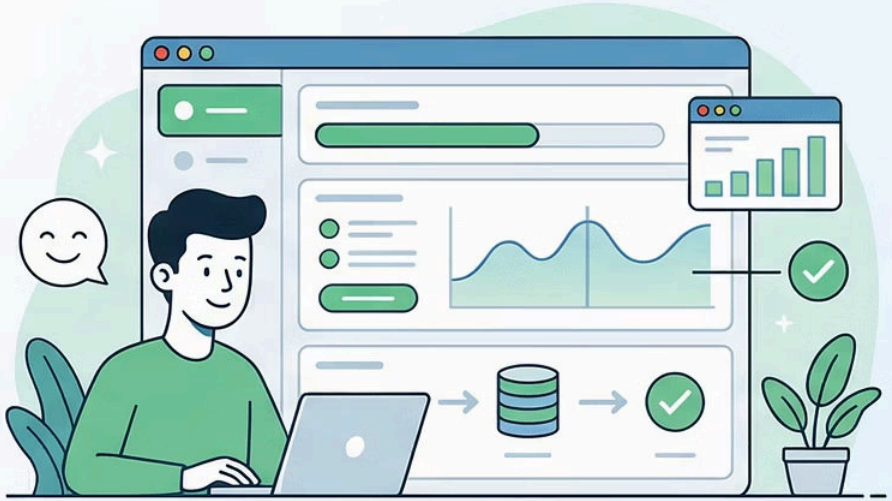
Valgrind界面显示堆内存分配状态，用不同颜色标注已分配、已释放、泄漏的内存块，
让程序员能够清晰看到每一次malloc和free的执行结果。



通过3D立体内存地图，原本抽象的内存地址空间变得具象可见。红色区域的内存泄漏不再是
隐藏的威胁，而是可以精确定位和修复的明确问题。

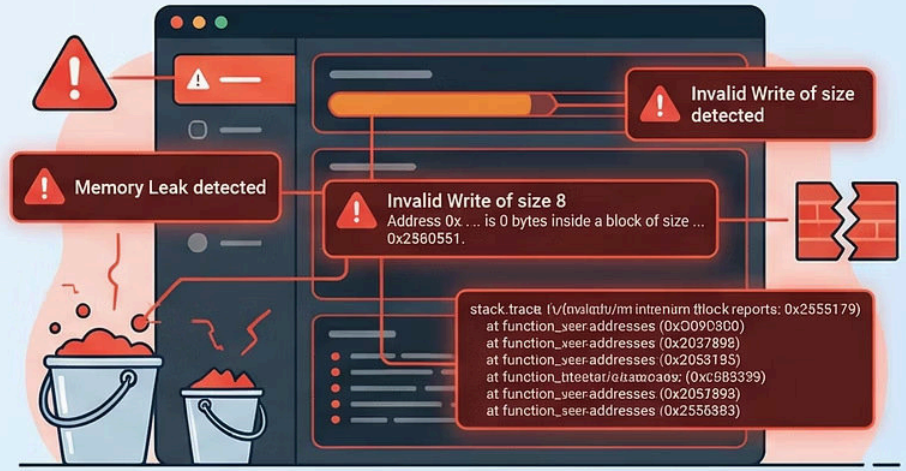
从隐蔽到可视化

运行表象



将隐蔽的内存泄漏转化为可视化的错误报告，这是Valgrind最重要的价值。程序表面运行正常，但内存正在悄悄流失。

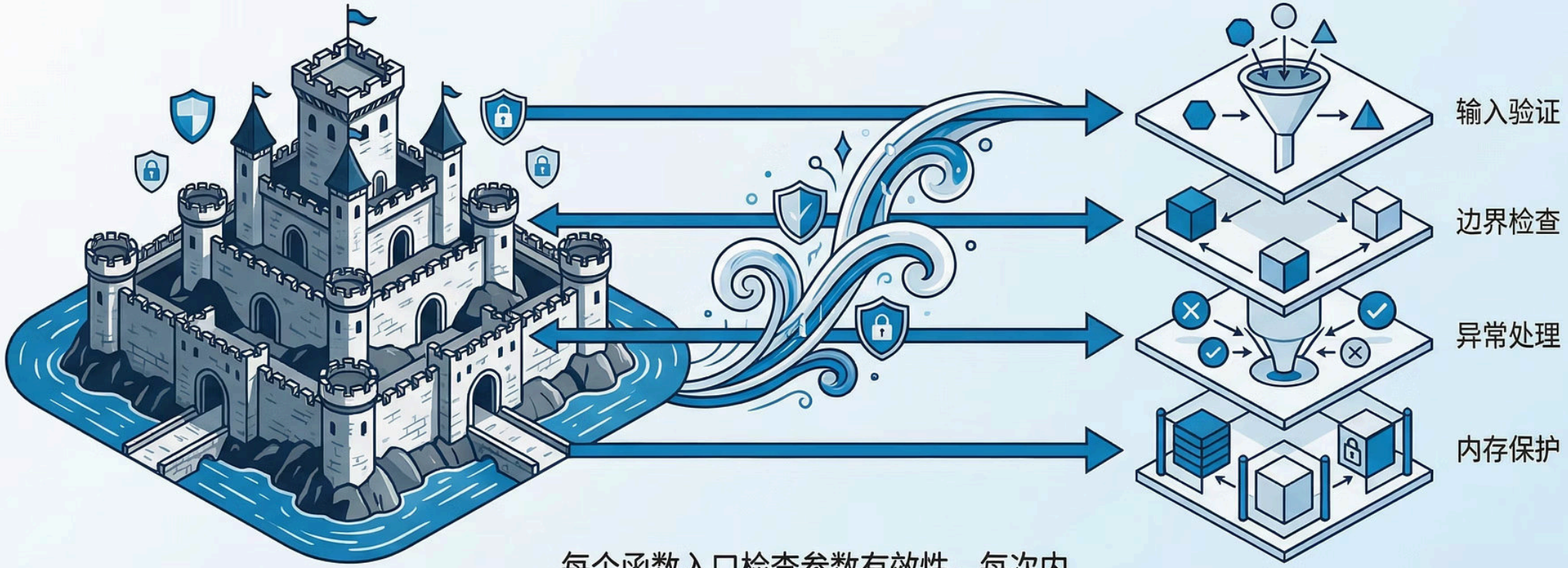
Valgrind揭示的真相



用户看到的是流畅的界面，Valgrind却发现了每秒数百次的微小泄漏。这些问题在短期测试中很难发现，但在生产环境的长期运行中会导致系统逐渐变慢直至崩溃。及早发现，及时修复，是专业开发的基本要求。

防御性编程的高阶心法

掌握防御性编程，从源头杜绝内存隐患。就像城堡建造者设计多层防御体系，程序员需要在代码中建立多重内存保护机制。



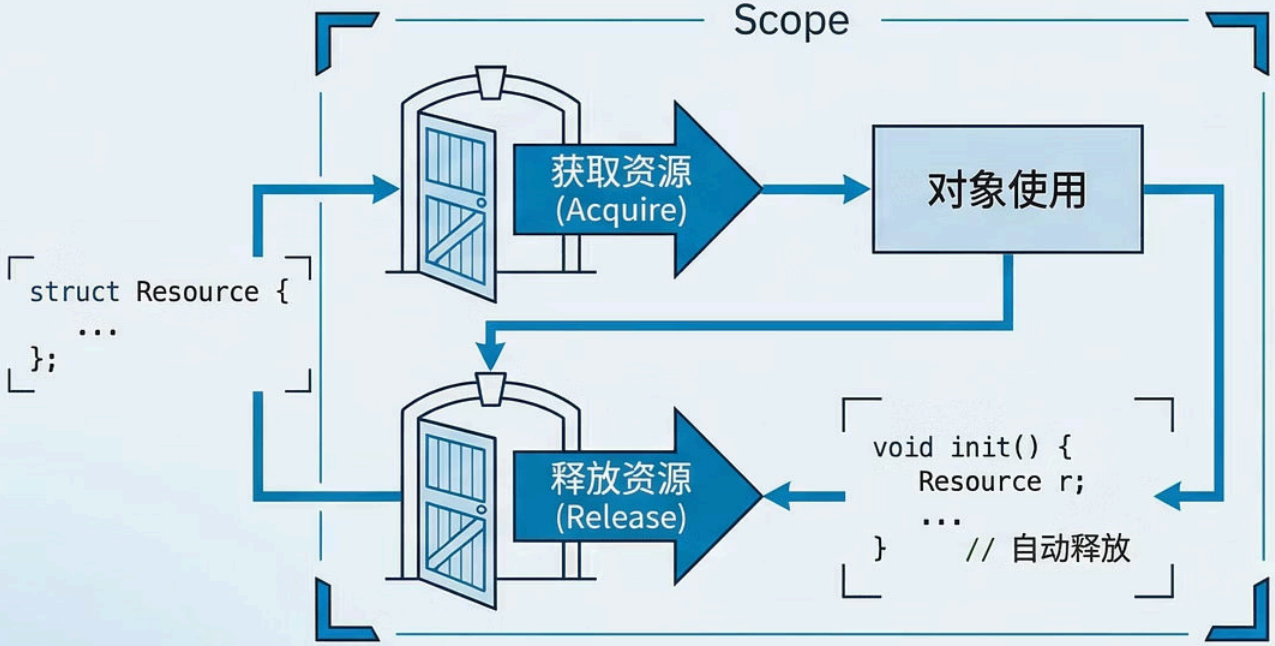
每个函数入口检查检查参数有效性

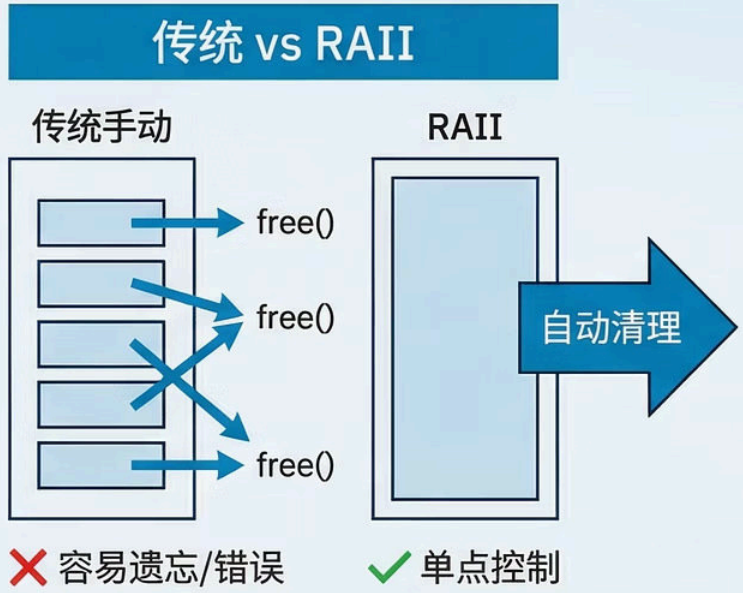
每个函数入口检查参数有效性，每次内存操作验证边界条件，每个资源分配都有到脑子剐都有对应的释放保障。

层层防护，确保即使某一处检查失败，系统仍能安全运行。

RAII：资源管理的艺术

RAII风格的资源管理封装，让内存管理变得优雅而安全。Resource Acquisition Is Initialization——资源获取即初始化，这个C++引入的概念在C语言中同样可以借鉴。





通过将资源生命周期与对象作用域绑定，消除了手动释放的负担和遗忘释放的风险。这是从手工作坊到工业化生产的飞跃，是程序设计思维的质的提升。

航天级代码的极端可靠性

航天级代码中.

航天级代码中.

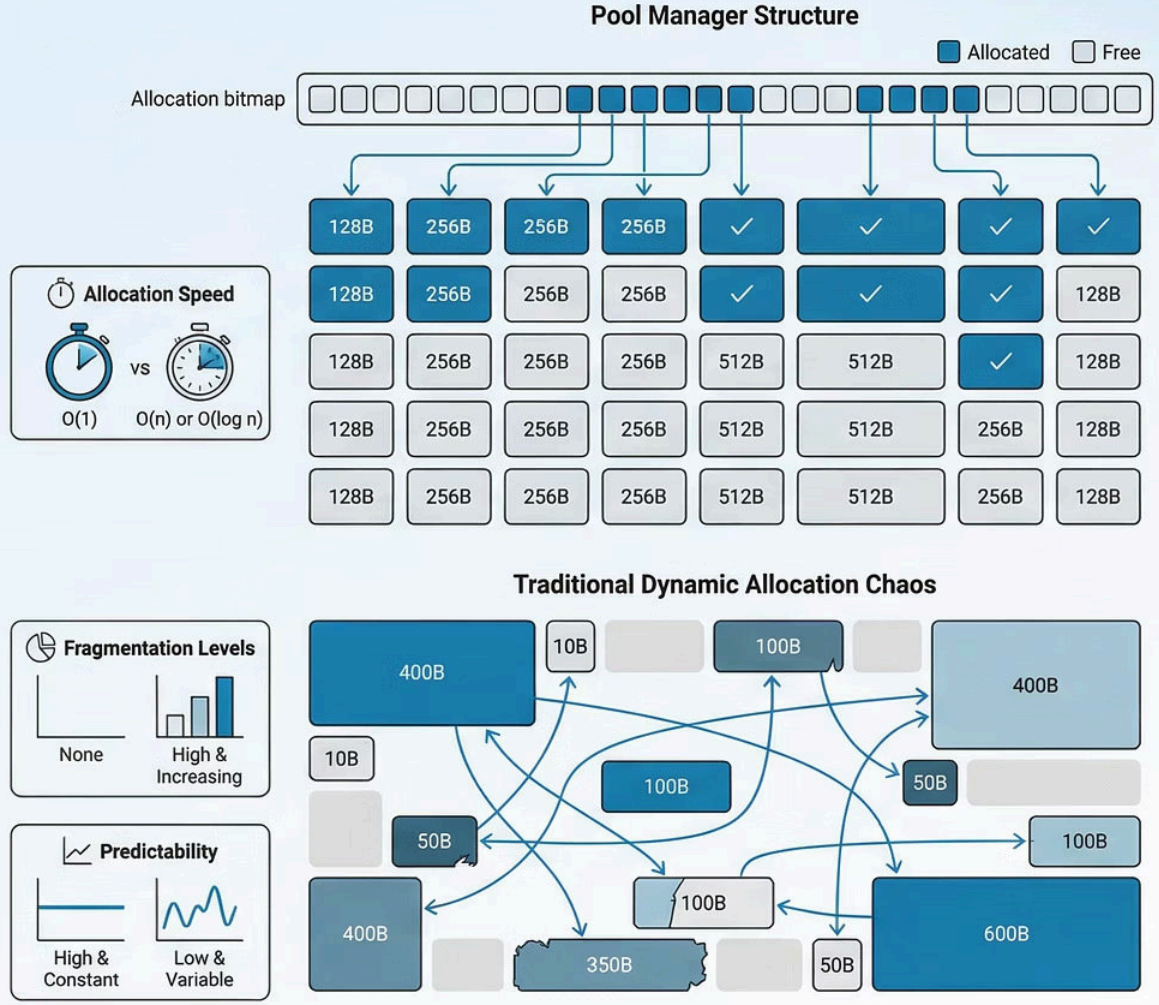
航天级代码中.



静态内存池的智慧

静态内存池的数据结构可视化，整齐排列列的的内存块像军队方阵一样规整，与传统动态分配的混乱状态形成鲜明对比。

预先分配固定数量的内存块，通过简单的位图管理分配状态。分配速度恒定O(1)，永不产生碎片，内存使用量完全可预测。这种设计牺牲了空间效率的适往灵活性，但在关键系统中，可预测性远比灵活性重要。

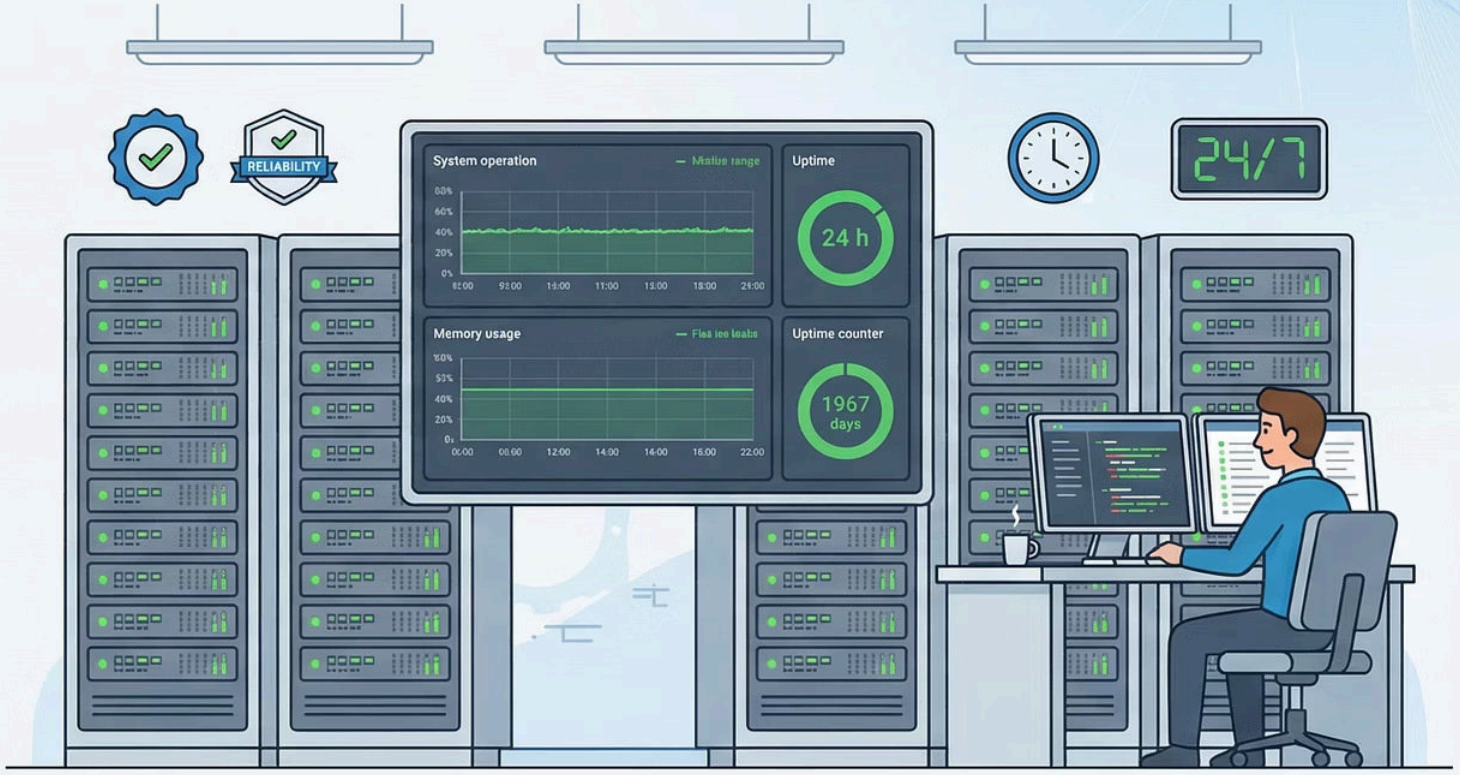


7×24小时压力测试

编写经得起7×24小时压力测试的工业级代码，这是从学生项目到生产系统的本质区别。

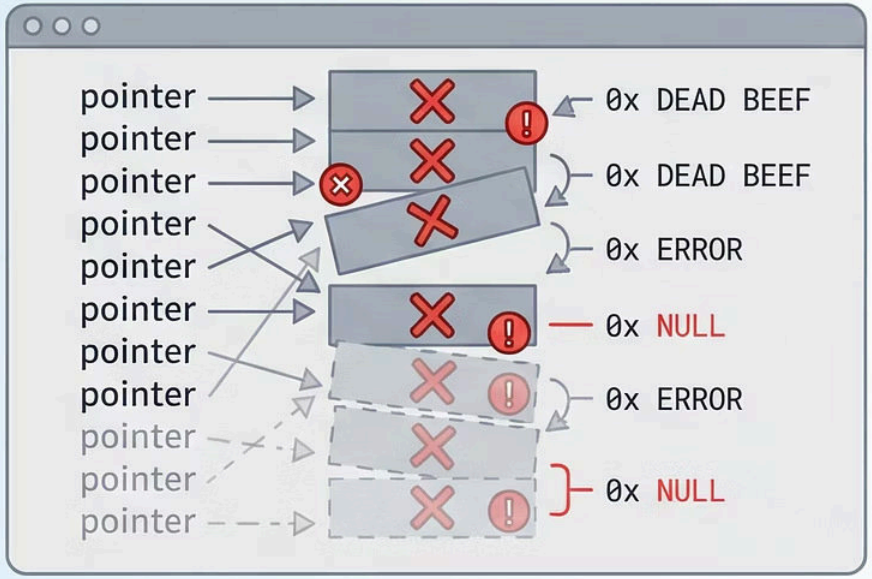
压力测试不仅要求程序功能正确，更要求在连续运行、高负载、边界条件下都能保持稳定。内存存使用曲线必须是平稳的直线，而不是缓慢上升的斜线。

能够在办公室演示成功是基础，能够在数据中心连续运行数月不出问题才是真正的工业级质量。

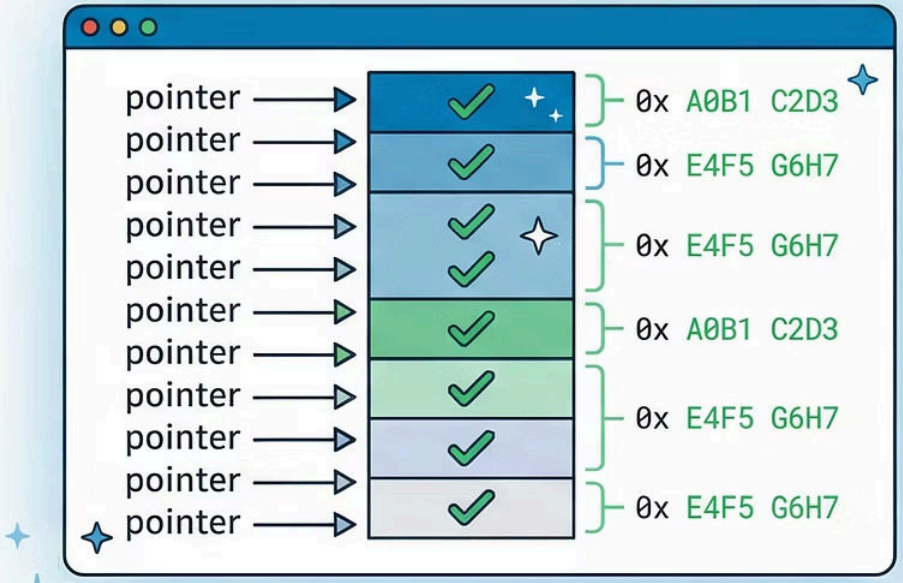


告别野指针与内存黑洞

修复前



修复后

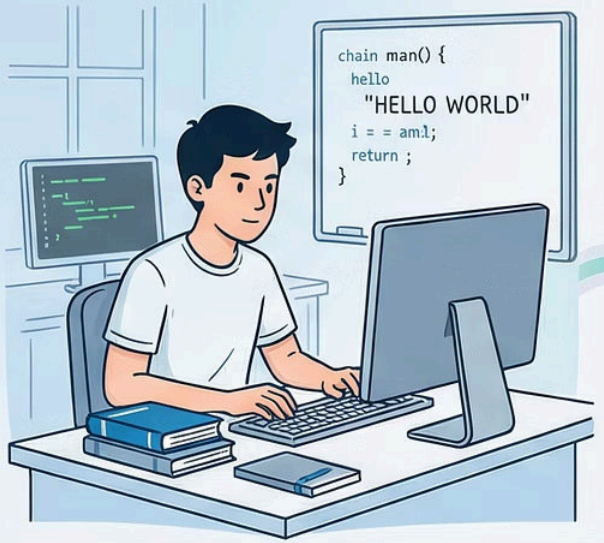


彻底告别。

内存黑洞。

从学生到工匠的蜕变

Basic Syntax



计算机专业学生在实验室认真编程，屏幕上显示通过所有内存测试的绿色提示，导师在旁边满意地点头。

Memory Management



这个蜕变过程不是一蹴而就的，而是通过无数次调试、修复、测试的循环积累而成。从写出来。

Professional Practices



这个软件工匠师创新运作工程师，并且等的项序通过程毕业就的开学提案。

精密代码，可靠未来

从火星探路者到天宫空间站，从3分钟噩梦到3000小时考验，内存管理的重要性已被无数案例证明。掌握Valgrind等专业工具，践行防御性编程理念，坚守职业伦理底线——这些不仅是技术要求，更是新一代软件工匠的必修课。每一行精密的代码，都在为更可靠的未来筑基。



用工匠精神铸就软件品质，以专业态度守护系统安全

三大核心支柱

工匠精神：对每一个细节的极致追求，对系统资源的珍惜敬畏，对长期稳定的责任担当



专业工具：Valgrind内存分析，静态代码检查，自动化测试框架，将隐患暴露在开发阶段

严谨实践：防御性编程，RAII模式，静态内存池，7×24小时压力测试标准。

三者结合，构筑起工业级软件的质量基石